```
  <button class="btn btn-dark" type="submit">Save</button>
</form>

{% endblock content %}
```

This template doesn't have anything you haven't seen before, except you have fresh eyes on `{% csrf_token %}`. Now, you should see it as an important piece of security technology rather than scary-sounding black magic.

7.2.5 Editing existing data with forms

You've got a form, and you've got a corresponding model in the database—all is good! Now, what if your user wants to make an edit? If they own the ad, they should be able to update its contents. With a few small changes to your code, you can handle this situation.

This scenario is commonplace. Most times when you have user driven input, you want the user to be able to edit the input. You could write two views, one for adding new content and one for editing existing content, but most of the code will be similar. Instead, you can write one view that works in two different modes.

Views are just functions, and like all functions, they can take default arguments. When in Edit mode, you need the view to take the ID of the item being edited. When in Add mode, you don't need an ID, but you do need some indication that you're adding instead of editing. ORM object IDs start counting at `1`, which means you can take advantage of this and use an ID of `0` to indicate Add mode. By setting the item ID's default value to `0`, you have two different ways of calling your view.

URLs aren't quite as flexible as Python functions. To go with your two-mode view, you can either use an optional query parameter for the item ID or register two different routes. The first route is for adding a new object, and the URL path contains no arguments. The Add route calls the view without any arguments, and the default ID of `0` gets used. The second route is for editing, and that URL takes the ID of the object to be edited.

The URL query parameter and two-route designs are equally effective. The latter is more Django-esque, though. Once you start using path arguments in a URL, going back to query parameters seems inelegant. This could just be my personal bias, though.

Let's put this into practice by adding the ability to edit ads in the `seeking_ad()` view. In this case, the object being edited is a `SeekingAd` model, which when you're in Edit mode, you'll need to fetch from the database. You've done this before, using the `get_object_or_404()` shortcut. With the object in hand, use the `instance` argument to prepopulate the `ModelForm` with the model's contents. This happens in both the `GET` and `POST` sections of the view when in Edit mode. Figure 7.6 shows how the original flow of `GET` and `POST` change now that the view handles both adding and editing objects. The code in listing 7.12 shows the changes to the view necessary for supporting both Add and Edit modes.